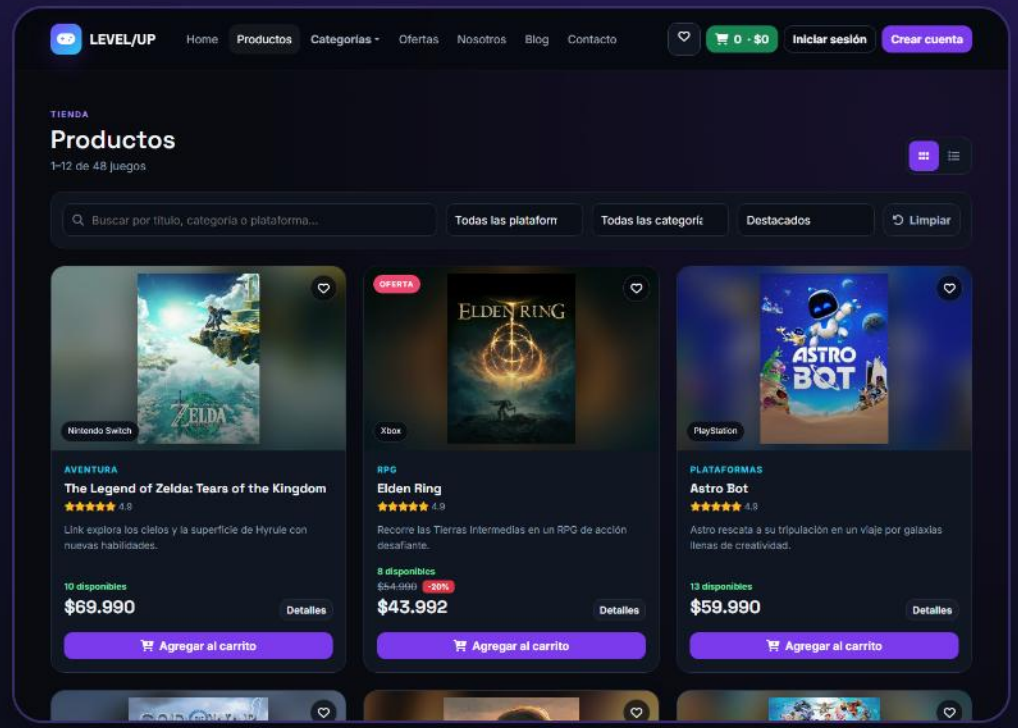
LEVEL/UP

Migración a React y pruebas unitarias

Tienda de videojuegos · Jasmine + Karma · Bootstrap 5

DSY1104 DESARROLLO FULLSTACK II · EVALUACIÓN PARCIAL 2

Max Quijada · Bastián Muñoz · Andrés Camilo Cruz



01 · DISEÑO CON UN FRAMEWORK JS

De HTML y JS a una SPA en React

Aspecto	Entrega 1 (legacy-v1)	Entrega 2 (React)
Vistas	Una página con modales	19 rutas de tienda y admin con React Router
Interfaz	innerHTML y funciones globales	Componentes reutilizables con props
Estado	Variables globales y persist()	useState, useReducer y Context
Datos	Catálogo leído de la API de RAWG	db.js con CRUD + localStorage, 48 juegos
Calidad	Sin pruebas	135 pruebas Jasmine + Karma, 90,9% cobertura

La versión original se conserva en la carpeta legacy-v1/ del repositorio

01 · DISEÑO CON UN FRAMEWORK JS

Arquitectura en capas

Componentes
pages/ y components/: GameCard, Navbar, Checkout...

→

Context + hooks
AuthContext, CartContext, useCollection, useForm

→

Servicios
authService, orderService, rawgService

→

db.js
CRUD por colección, persistido en localStorage

Tiempo real

Cada escritura en db.js avisa a sus suscriptores; useCollection vuelve a renderizar la vista.

Responsabilidad única

Componentes que solo muestran, separados de la lógica pura y de los datos.

Fácil de cambiar

Para usar el backend real solo se reemplaza db.js por fetch; los componentes no cambian.

01 · DISEÑO CON UN FRAMEWORK JS

Navegación con React Router

Tienda (pública)

- Home · Productos → Detalle
- Categorías → Categoría · Ofertas
- Carrito → Checkout → Pago correcto o Pago con error
- Login → Registro · Mi cuenta · Favoritos
- Nosotros · Blog → 2 artículos · Contacto

Administrador (/admin)

- Dashboard · Órdenes → Boleta
- Productos → Nuevo, Ver, Editar, Críticos, Reportes
- Categorías → Nueva, Editar
- Usuarios → Nuevo, Ver, Editar, Historial
- Reportes · Perfil

ProtectedRoute cuida el panel: sin sesión redirige a /login y un cliente vuelve al Home.

02 · COMPONENTES Y RESPONSIVO

Props: componentes que solo reciben datos

```
// components/store/GameCard.jsx
export default function GameCard({
  game, onAddToCart, onToggleFavorite,
  isFavorite = false, layout = 'grid'
}) {
  // …muestra título, precio y stock
  <button disabled={game.stock ⩽ 0}
    onClick={() ⇒ onAddToCart(game)}>
```

Presentacional

GameCard no conoce el carrito: recibe el juego y la acción por props.

Contenedor

GameGrid conecta GameList con useCart y useFavorites.

Reutilizable

La misma tarjeta sirve en Home, Catálogo, Ofertas y Favoritos.

02 · COMPONENTES Y RESPONSIVO

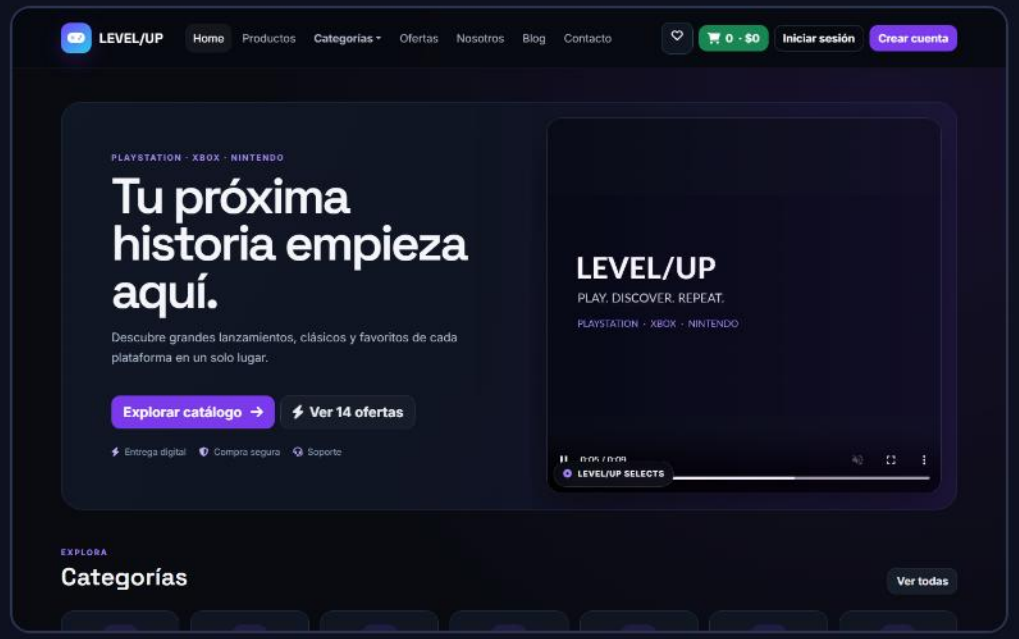
Estado: local, global y persistido

```
// context/CartContext.jsx
export function cartReducer(state, action) {
  case 'add': {
    const next = Math.min(stock,
      current + quantity);
    return { ...state, lines: [...] };
  // remove, setQuantity, applyCoupon, clear
```

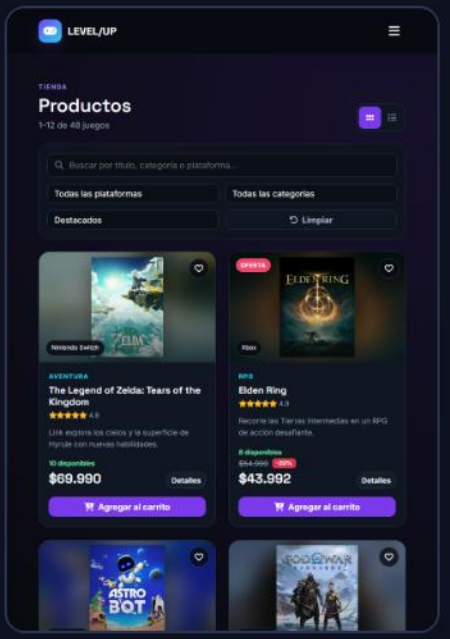
Tipo	Dónde
useState	Filtros, página, cantidad, modales
useForm	Valores y errores de cada formulario
useReducer	Carrito: acciones puras e inmutables
Context	Sesión, carrito, favoritos y avisos
localStorage	Persistencia entre recargas

02 · COMPONENTES Y RESPONSIVO

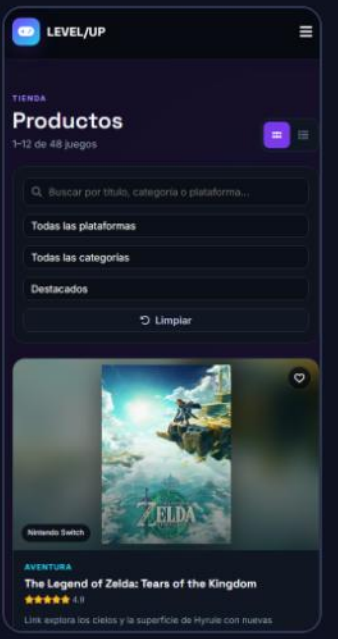
Diseño responsivo con Bootstrap



Escritorio · 3 columnas



Tablet · 2 columnas



Móvil · 1 columna

`row-cols-1 row-cols-sm-2 row-cols-xl-3 · navbar-expand-lg · col-md-6`

03 · PRUEBAS UNITARIAS

Resultado de npm test

135

pruebas exitosas

0

fallos, en unos 2,5 segundos

90,9%

cobertura de statements (mínimo 70%)

03 · PRUEBAS UNITARIAS

10 pruebas, una por cada categoría

#	Categoría	Qué verifica	Archivo
1	Render de lista	GameList dibuja los 48 juegos	store.spec
2	Condicional	AlertMessage aparece solo si hay error	common.spec
3	Props	Button recibe la etiqueta y el onClick	common.spec
4	Props	GameCard sin stock deshabilita la compra	store.spec
5	Estado	El formulario de Login cambia al escribir	auth.spec
6	Estado	cartReducer nunca supera el stock	cartReducer.spec
7	Evento	Un clic en Agregar llama a onAddToCart	store.spec
8	Mock	authService lanza un error y se ve la alerta	auth.spec
9	Mock	Pago rechazado: no descuenta stock	shopping.spec
10	Mock async	fetch de RAWG: OK, error 500 y sin red	rawgService.spec

03 · PRUEBAS UNITARIAS

Anatomía de una prueba

```
// pages/store/auth.spec.jsx
it('muestra el error del servicio', () ⇒ {
  spyOn(authService, 'login')
    .and.throwError('Correo o contraseña…');
  const { container } = renderApp('/login');
  fill(container, 'email', 'user@game.com');
  fireEvent.click(botonIngresar);
  expect(screen.getByRole('alert')
    .textContent).toContain('incorrectos');
});
```

1 · Preparar

Un spy reemplaza el servicio real y fuerza el error.

2 · Actuar

Se renderiza la ruta y se simula al usuario.

3 · Verificar

expect revisa lo que el usuario ve en el DOM.

04 · PROCESO DE TESTEO

Configuración del entorno de pruebas



Aislamiento

beforeEach limpia localStorage y reinicia la BD; afterEach desmonta los componentes. Ninguna prueba depende de otra.

Barrera de calidad

Umbrales mínimos: 70% statements, 55% branches, 65% functions y 70% lines.

04 · PROCESO DE TESTEO

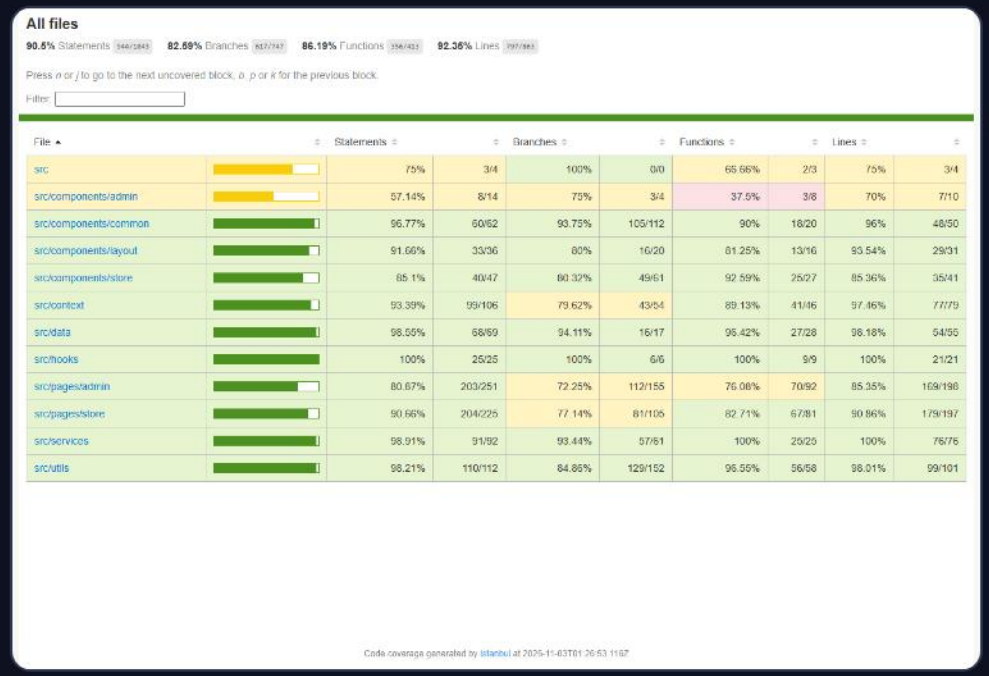
Mocks: aislar lo que no controlamos

Dependencia	Técnica de Jasmine	Para qué
Pasarela de pago	`and.returnValue(...)`	Forzar pago aprobado o rechazado
fetch (API RAWG)	`and.callFake(...)`	Simular OK, error 500 y sin red
authService	`and.throwError(...)`	Probar mensajes de error
db.games.create	`and.callThrough()`	Revisar argumentos del CRUD
Callbacks	`jasmine.createSpy()`	Verificar que un evento se emite

Los servicios son **objetos**: spyOn puede reemplazar sus métodos, cosa que no permite un export suelto de un módulo ES.

04 · PROCESO DE TESTEO

Análisis de resultados



All files

90.6% Statements 944/1043 **82.68%** Branches 617/747 **86.19%** Functions 356/413 **92.36%** Lines 797/863

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
src	75%	3/4	100%	0/0	66.66%	2/3	75%	3/4
src/components/admin	57.14%	8/14	75%	3/4	37.5%	3/8	70%	7/10
src/components/common	96.77%	60/62	93.75%	105/112	90%	18/20	96%	48/50
src/components/layout	91.66%	33/36	80%	16/20	81.25%	13/16	93.54%	29/31
src/components/store	85.1%	40/47	80.32%	49/61	92.59%	25/27	85.36%	35/41
src/context	93.39%	99/106	79.62%	43/54	89.13%	41/46	97.46%	77/79
src/data	98.55%	68/69	94.11%	16/17	96.42%	27/28	98.18%	54/55
src/hooks	100%	25/25	100%	6/6	100%	9/9	100%	21/21
src/pages/admin	80.67%	203/251	72.25%	112/155	76.08%	70/92	85.35%	169/198
src/pages/store	90.66%	204/225	77.14%	81/105	82.71%	67/81	90.86%	179/197
src/services	98.91%	91/92	93.44%	57/61	100%	25/25	100%	76/76
src/utils	98.21%	110/112	84.86%	129/152	96.55%	56/58	98.01%	99/101

Code coverage generated by istanbul at 2026-11-03T01:26:53.116Z

coverage/html/index.html · cobertura por carpeta

Fuerte: lógica de negocio

Servicios, datos y hooks entre 97% y 100%.

Brecha: detalles visuales

Menú móvil del admin (57%) e impresión de la boleta.

Bug encontrado por las pruebas

Con portadas locales no se podía editar ningún producto; se corrigió con una prueba de regresión.

¿Preguntas?

REPOSITORIO

github.com/max-alexander/levelup-react

EJECUTAR

```
npm run dev
npm test
```

CUENTAS DE PRUEBA

admin@game.com
user@game.com · 123456